AR Pet Horse – By Laiba Siddique on 12/15/25

Project overview and goals

My augmented reality (AR) project was an interactable AR pet horse. In this app, the player can spawn in an animated, low poly pet horse model and perform a few interactions with it. The goal of this app is to create a short, little, sweet simulation of a pet in AR, and the chosen pet for this was a horse. The app allows people who are unable to own a real pet, such as a horse, to have a virtual alternative, similar to how Pokémon and Tamagotchi allowed people to have virtual pets, except the pet horse appears in the player's real space on their screen.

To spawn in the horse, upon opening the app, the player is presented with a bar of text explaining that to spawn the horse, they have to tap a plane. To do this, the player has to start moving their device and camera around so the app can see and detect horizontal surfaces, upon which blue planes will appear. On a blue plane, the user taps where they would like to spawn the horse. When the app starts, the main UI will also be visible, but the buttons will not do anything as there is no horse for them to interact with yet.

The player can interact with the horse using the main UI buttons. The player can press/tap the "feed" button to feed the horse and have it eat from its food bowl, which increases its energy. The "feed" button also causes an animation of the horse eating and a short affectionate horse sound to play. When the horse eats, particles emit from the food bowl, indicating the horse it is eating, especially a little messily. The player can also tap the "run" button to have the horse run for a bit, decreasing its energy. The "run" button also causes an animation of the horse running, and a horse running sound to play. Pressing the "info" button shows/hides the UI presenting the horse's energy. The horse cannot run when it has no energy (i.e. energy <= 0). Feeding the horse gives it 1 energy, and having it run makes it lose 1 energy. The player cannot feed the horse while it is running. The player cannot have the horse run while it is eating either. This is enforced by disabling both the run and feed buttons until the actions of the pressed button end. Lastly, pressing the "speak" button makes the horse neigh, and hence "speak" (similar to how people command their pet dogs to "bark").

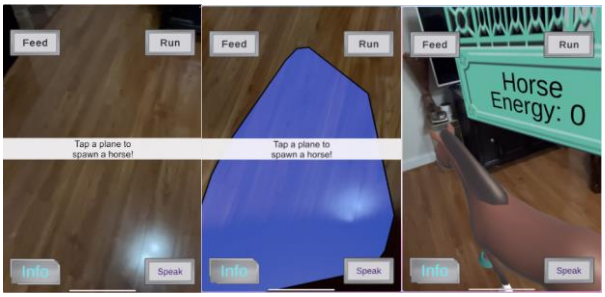


Image 1: Starting UI Instructions and Main UI (First thing that appears when game starts); Image 2: Blue Plane Appearing Before Horse is Spawned In; Image 3: Horse Spawned-In and Horse Energy UI Being Shown

I developed the AR Pet Horse app on my iPad Air (6th Gen; 2024 M2 chip; iPadOS 18.2), MacBook Pro (M3 Pro chip; 2023; MacOS Sequila (15.5)), and Unity (2022.3.42f1). The app is directed towards iPad Air M2 chip; however, I think it would work on most other AR supporting iPads and iPhones too as it is not too intensive. I also used Maya 2025 to aid in making a few assets. **I did not have access to any Android device, so I was unable to generate and submit an APK file, so it is missing from the submission**. (Also, even if I could generate an APK file, I would not be able to validate that the app works on Android without an Android device. Hence, this app is for iOS and iPadOS only.).

Design process and decisions with figures and drawings

The design process for AR Pet Horse started with a project proposal. In the proposal, I chose this animal pack from the Unity Asset store that included models and a few premade animations for each. In the proposal, the AR Pet Horse app was originally supposed to be an AR Pet app with multiple pet options. However, due to time constraints and the horse model being the only animal asset in the pack with a premade eating animation, I decided to only simulate a pet horse and change the app to being the "AR Pet Horse" app instead. For the UI and interactions, I had planned in the proposal that the app would have the info, feed, run or walk buttons, a dropdown for choosing which pet to have, and the ability to tap on the pet to "pet" it and make it play a sound indicating it had been pet. However, due to time constraints, I was unable to implement the pet functionality and replaced the dropdown with a "speak" button instead. For the "speak" and other animal sounds, I did not plan ahead where I would get them but ultimately downloaded this pack of sounds from the Unity Asset store. I had also planned the run and feed system mentioned in the project overview above within the proposal. However, in the proposal, the info panel was planned to show how many times the pet had eaten and how many times they had run. I changed this to only showing overall pet energy in the app because it

was less wordy, easier to understand and explain to users, more realistic, and seemed like it would be easier to implement this way.

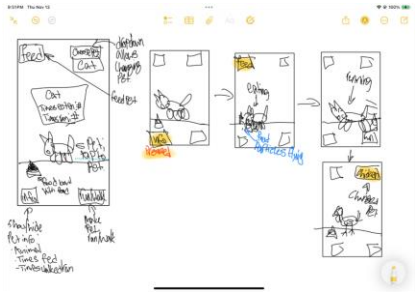


Image 4: Proposal Storyboard Sketch of Original Plan of App

In the proposal, we were also asked to explain how different aspects of the app fit the project requirements. I met the 3D accessories requirement downloading by downloading the horse asset from the Unity Asset store, creating the food and food bowl assets by myself in Maya, and editing the fire particle system from the Unity VR tutorial and my VR project to be repurposed as food particles. The food asset was actually the ash asset I created for my VR project, but slightly recolored and without the ash texture to repurpose it as food. I added the food and food bowl assets to the horse prefab to force them to spawn with the horse in a set location to the horse and become a part of the main horse asset. The horse model did not need optimization since it was already low poly, and the app already ran smoothly on my iPad without further optimizations. The food particles fulfilled the particles requirement.

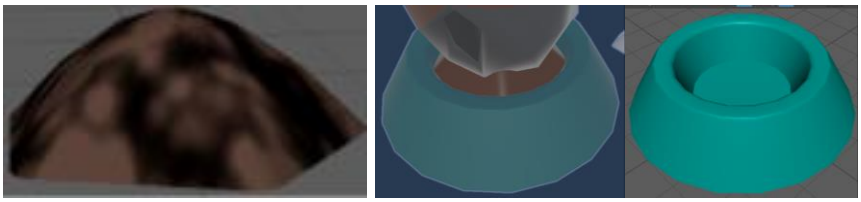


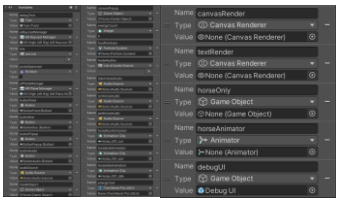
Image 5: Ash Asset in Maya; Image 6: Food Bowl with Food in Horse Prefab; Image 7: Bowl Asset in Maya

I met the animation requirement by having the horse play an “idle” animation whenever it was not running or eating, play an “eating” animation when eating, and play a “running” animation when running. I also met the environmental accessories requirements by placing the food bowl with food in front of the horse when it spawns and by implementing the “feed and run based on energy” to create an immersive and realistic interactive experience for the user. This is because, realistically, all animals need to eat, pets often eat from their designated food bowl, eating causes an increase of energy (as long as it is healthy), and running uses up energy. Additionally, the energy panel accessory allowed users to observe, learn about, and understand

this energy interplay without having to guess why the horse could not run at certain points. Lastly, I met the user interaction requirement by allowing the user to feed the horse, have it run, have it eaten, and being able to view the horse's energy through UI button taps.

Technical implementation details

I reused and repurposed the AR Markers and AR Planes homework assignments for this project. The main UI, audio/speak button, popup functionality, variable initialization setup, and very base of the run and feed functionality (i.e. both are related, so they can be put in the same file and how to use on button press) was taken from the AR Marker homework. The tap to place functionality and starting instructions UI bar was taken from the AR Plane homework. Some of the variables I used were from those homework assignments; however, I also used many other variables too.



Images 8-10: All Visual Scripting Variables Created

For the visual scripting, I started with the scripts from the homework assignments, edited them to my use case, and created my own. In the "tap to place" visual script, I changed it to spawn the horse prefab instead of the portal. In the "setup" script, I expanded it heavily to initialize all the variables I needed, including but not limited to the horse itself, the popup, the text on the popup, a list of the audio sources in the horse and then the audio source variables themselves, the particles, and the horse animator. In the "play audio" script, which activates when the "speak" button is pressed, I changed it to play a horse sound instead of waves. I did not have to change the "display popup" script because the popup in this project and the AR Marker homework was assigned to the same variable name.

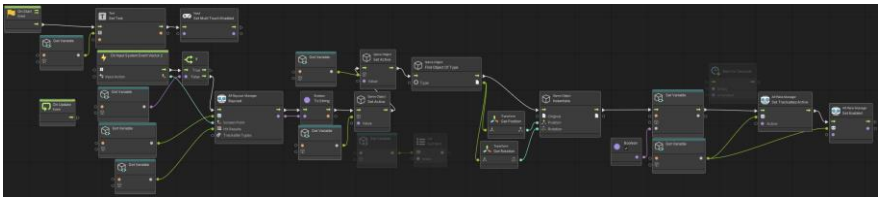
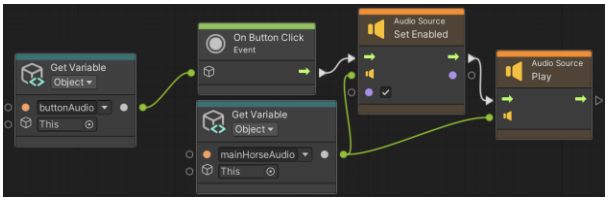


Image 11: Tap to Place Script



Image 12: Setup Script



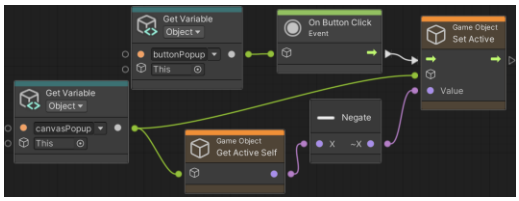


Image 13: Play Audio Script; Image 14: Display Popup Script

For the “run” and “feed” button functionalities, I implemented their functionalities in the “Run and Feed” visual script. This is where the majority of my code (outside of setup) was placed. I created an integer variable named “energyCount” and initialized it to 0 in the variables column. When the “feed” button is pressed, it increments energyCount by 1 and updates the energy text on the popup to reflect the new value to show the horse’s energy is increased. Then, the “run” and “feed” buttons are disabled to prevent the user from feeding the horse when it is running, and have the horse run while it is eating, and avoid any animation, sound, variable, text, and other conflicts that could occur by tapping the other button or re-tapping the button that was just pressed while the button’s functionality is still active. Then, the eating audio, eating animation, and particles are started. After waiting for 5 seconds for the sound and animation to finish, the particles are deactivated, idle animation is retriggered, which is waited on for 1 second to allow for a smooth transition before any other animation can be played, and the “run” and “feed” buttons are reactivated.

When the “run” button is pressed, it checks if energyCount is greater than 0 (so if the horse has energy). If not, then the button does nothing. Otherwise, it decrements energyCount by 1 and updates the energy text on the popup to reflect the new value to show the horse’s energy is decreased. Then, the “run” and “feed” buttons are disabled for the same reasons as the “eat” button did. Then, the running audio and animation are started. The running animation has to be played multiple times, with short delays in between to prevent the next trigger of the animation from interfering with the previous trigger, since it is short and to better simulate the action of the horse running. After waiting 10 seconds for the sound and animation to finish, the “run” and “feed” buttons are reactivated (the idle animation is not retriggered because I realized the horse goes back to it automatically). The “speak” and “info” are never disables because horses can

neigh when eating and running, and the user should be able to show or hide the horse info whenever they want since it does not interfere with the run and feed functionalities.

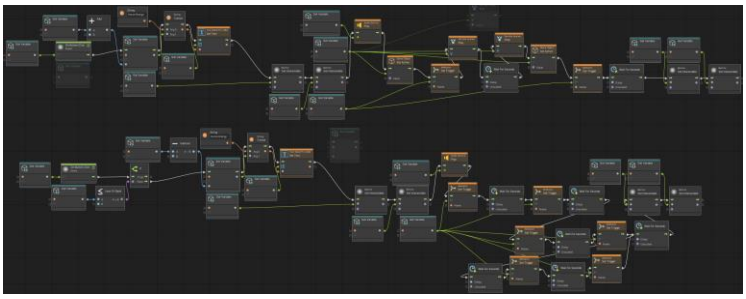


Image 15: Run and Feed Script

For the horse prefab and animations, I had to make some changes to the downloaded asset. First, I had to add my own food and food bowl assets, the popup asset, and running, and start eating, and running audios to the horse prefab. The "start eating" audio plays when the horse starts eating and is named "PetAudio" since it was originally supposed to play when the horse was pet, an interaction I ran out of time to implement. The "speak" audio is just named "audio" because I reused the variable from the AR Markers homework and just changed the audio it plays. The horse prefab was already in the eating position, so positioning the food and its bowl to the location the horse would eat from was easy. I positioned the "start eating" and "speak" audios in the location the horse's head would approximately be when idle, and the "running" audio to be at its feet to simulate where sounds come realistically. The horse animator originally used a blend tree. However, after multiple attempts, I was unable to figure out how to use it, especially in visual scripting, so I changed the animator to use multiple transition states and set up my own transitions and triggers instead.

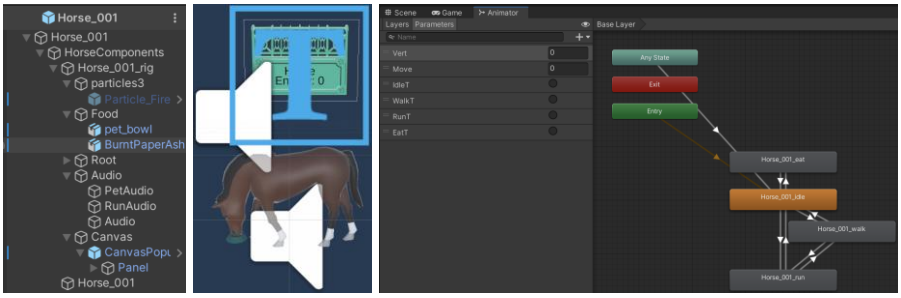


Image 16: Horse Prefab Hierarchy; Image 17: Horse Prefab; Image 18: Horse Animator

Challenges faced and solutions

I faced multiple challenges in this project. The first challenge came from the horse prefab. Whenever I ran the app, the panel would be at the horse's side, and the food and bowl would be

much farther to the side and below the horse (the original area they were all spawned in when inserting into the prefab). To try to fix this, I placed the food, pet bowl, and particles in an empty parent that was centered at (x, y, z) = (0, 0, 0) before adding the assets. I also placed the audios and panel in their respective parents centered at (x, y, z) = (0, 0, 0). This did not solve the entire problem though. Eventually, I realized I should try to place all these assets in the horse rig rather than just the horse components, which fixed the issue. However, further testing showed that the food particles (named "Particle_fire" in the prefab) were never showing up, even though a new instance of particles outside the horse did. I tried placing the particles in a separate parent from the food and even tweaking the particle settings multiple times to try to fix it, but those did not work. Eventually, I decided to just put in a new instance of particles into the prefab because it seemed that instance already there was buggy and had some sort of weird setting or bug preventing them and their copies from showing. *(See image 16 for end hierarchy)*. Another challenge related to the horse was already mentioned above, in which I could not figure out how to animate the horse using the provided blend tree, so I changed the animator to use transition states instead.

The most difficult challenge I encountered was that the horse would always fall through the floor when spawned in, and spawn in the camera location rather than where the plane was tapped. I tried adding rigid body with gravity off and continuous dynamic collision and kinematic on and adding a mesh collider with convex on, but that did not fix the issue. Further testing showed that removing the plane was what caused the horse to fall, so I kept the plane in during most of testing since keeping it in also had the horse spawn close to where I tapped. I thought maybe the horse spawns in after the plane disappeared, which may have caused it to spawn in the camera and then fall through. So, I tried adding a short delay after the horse spawned in before removing the plane. The horse still fell through as soon as the plane was removed. Eventually, after many hours of trial and error, I decided to look into the provided movement scripts that were attached to the prefab, which allowed for simulating realistic animal movements in games and controlling the animal with a mouse and keyboard, and realized they turned gravity on for the horse even without rigid body gravity on. When I disabled the script components on the horse, the horse finally spawned in the correct tapped location and did not fall through the floor when the plane was removed.

Future improvements and potential applications

AR Pet Horse can be improved by including functionalities that were missing from the final submission. For example, I can implement the “tap pet ot pet it” functionality to improve the user experience and make the app more realistic and immersive. I can also implement the ability to have multiple different kinds of pets and swap between them using a drop-down menu to bring the app back to its original plan of being an AR Pet app instead of just for a horse. I can also make the instruction interface a bit clearer as it may be confusing to some users and not have the buttons UI appear until the pet is spawned. Lastly, I can improve the app so the pet goes through an entire interactive lifecycle, similar to Tamagotchi.

A potential application of AR Pet Horse is as a pet simulator game, especially if all the above improvements are implemented. Another potential application is for teaching prospective pet owners some possible interactions that can be done with pets and the energy interplay within pets to help them become better pet owners later on.